MARKETLENS (MARKETDATA) - INTERVIEW HANDOUT
Generated: 2026-05-22

## 1) 30-SECOND PROJECT PITCH

I built MarketLens, a Java 21 + Spring Boot market data platform that ingests daily stock candles from Alpha Vantage, stores normalized data in PostgreSQL, computes technical indicators (RSI/MACD), and serves both API and dashboard UX surfaces. The core engineering focus was reliability under third-party API limits: idempotent pipeline runs, retry/circuit-breaker resilience, data-quality checks, and quarantine of malformed provider rows.

## 2) PROBLEM I WAS SOLVING

- Need dependable market-data ingestion despite provider throttling, malformed payloads, and retry scenarios.
- Need analytics-ready endpoints (indicators, adjusted prices, quality reports) for consumers.
- Need production fundamentals: API security, observability, and operational runbooks.

## 3) WHAT I BUILT (SCOPE)

- Backend: Spring Boot 4.0.1, Java 21, Spring Data JPA, Spring Security.
- Database: PostgreSQL with Flyway migrations.
- External provider integration: Alpha Vantage client.
- Analytics: RSI, MACD, split-adjusted prices, market calendar support.
- Reliability: idempotent run ledger, retry + backoff + circuit breaker, per-row quarantine for bad records.
- Platform controls: API key auth, role-based authorization, rate limiting, per-key daily quota, provider quota tracking.
- Observability: structured JSON logs, request IDs, Prometheus metrics, OTLP tracing.
- UI: static dashboard pages for watchlist, indicators, runs, quality, actions, calendar, status, and key administration.

Concrete repository indicators:
- 27 mapped API endpoints in controllers.
- 8 Flyway migrations.
- Service modules split by domain: ingestion, indicator, market, calendar, quality, retention.

## 4) ARCHITECTURE TALK TRACK (HIGH LEVEL)

Request path:
Client/UI -> API key filter -> per-key quota + global rate limiting -> controller -> service layer -> repositories/external provider.

Separation of responsibilities:
- Controllers: thin HTTP boundary.
- Services: business and resilience logic.
- Repositories: persistence and upsert/query concerns.
- Client adapters: external API integration and transport-level resilience.

Main reliability boundary:

- Ingestion run metadata is persisted in pipeline_run and keyed by optional
  Idempotency-Key, so retries are safe and auditable.

## 5) IMPORTANT ENGINEERING DECISIONS I CAN DEFEND

A) Idempotent ingestion runs
- pipeline_run stores run_type, statuses, retries, requested window,
  started_by, and unique idempotency_key.
- Benefit: safe retries/backfills without duplicate run records.

B) Degrade gracefully instead of hard-failing on dirty upstream data
- Malformed or invalid rows are written to ingestion_quarantine with reason,
  payload, source, and run_id.
- Benefit: run keeps progressing while preserving forensic traceability.

C) External API resilience
- Alpha Vantage calls use retry with backoff and a circuit breaker.
- Retryability is explicit (e.g., throttle/server errors retryable,
  semantic API errors non-retryable).

D) Quota strategy at two layers
- Provider quota (Alpha Vantage daily cap) tracked in DB.
- Consumer/API-key quota tracked per key, per day.
- Benefit: prevents both upstream overuse and downstream abuse.

E) Data model scaled for time-series growth
- price_candle partitioned by trade_date range (yearly partitions + default).
- Benefit: keeps performance predictable as historical rows grow.

F) Compute/caching balance on read paths
- Calendar and adjusted-price responses are cacheable.
- Benefit: lower repeated computation and lower DB load on hot ranges.

## 6) END-TO-END INGESTION FLOW (WHAT HAPPENS ON RUN)

1. Create pipeline_run (RUNNING) with metadata and expected symbol count.
2. Resolve symbol set from active watchlist (or request payload for backfill).
3. For each symbol:
   - consume one provider quota call;
   - fetch candles from provider;
   - validate row-level shape/values;
   - quarantine invalid rows;
   - upsert valid rows idempotently;
   - recompute indicators.
4. Aggregate processed/failed/retry counts.
5. Persist final status: SUCCESS, PARTIAL, or FAILED.

## 7) OPERATIONS & PRODUCTION READINESS

- Health endpoints available for liveness checks.
- Prometheus metrics exposed (admin-gated).
- Structured logs include request context and API key fingerprint.
- Suggested SLO posture documented:
  - API availability: 99.9% monthly
  - p95 latency: < 500ms for read paths
  - ingestion success rate: > 99% daily

- Runbook includes quota-exhaustion and migration troubleshooting.

## 8) SECURITY POSTURE SUMMARY

- API key required for /api/** (except documented public paths).
- Role-based access:
  - USER for standard data endpoints
  - ADMIN for key administration and Prometheus metrics
- Unauthorized returns 401 Problem JSON.
- Quota/rate violations return 429 with limit headers.
- API key fingerprinting in logs improves auditability without exposing raw secret material.

## 9) TRADEOFFS AND WHAT I WOULD IMPROVE NEXT

Current tradeoffs:
- Per-key quota registry is currently in-memory; a restart resets usage.
- Generated API keys are runtime-managed and not persisted for long-lived multi-instance coordination.
- Minimal automated tests currently in repository.

Priority next steps:
1. Persist app-key quota/metadata in PostgreSQL or Redis for horizontal scale.
2. Add integration tests for ingestion idempotency, retries, and quarantine.
3. Add scheduled ingestion orchestration and dead-letter replay tooling.
4. Expand indicator set and optimize recompute strategy.
5. Add stronger key lifecycle controls (expiry, revocation history, audit log).

## 10) INTERVIEW Q&A QUICK ANSWERS

Q: "How did you handle third-party instability?"
A: Retry with backoff + circuit breaker + explicit retryable exception model; malformed payload rows are quarantined so one bad record doesn't fail a run.

Q: "How did you prevent duplicate processing?"
A: Idempotency-Key support with unique index on pipeline_run.idempotency_key, plus idempotent upsert behavior for candle data.

Q: "How does your design scale over time-series growth?"
A: Year-partitioned price_candle table and indexed symbol/date access paths, with cache on heavy read computations.

Q: "How do you monitor and operate it?"
A: Health + Prometheus + structured JSON logs + request IDs + documented SLO targets and alert examples.

Q: "What would you improve first in production?"
A: Persist per-key quotas and generated key metadata, then strengthen test coverage around ingestion correctness and resilience paths.

## 11) 90-SECOND STORY VERSION

I designed MarketLens as a resilient market-data pipeline rather than just an API wrapper. The hard part was not fetching candles, it was handling reality:

provider throttling, partial failures, and messy upstream rows. I modeled pipeline runs explicitly with idempotency, so retries and backfills are safe and auditable. On each symbol ingest, I validate every candle row, quarantine bad payloads for follow-up, upsert valid data, and recalculate indicators.

On the serving side, I focused on practical production controls: API key auth, role-based admin access, rate limiting, quotas, observability, and operational runbooks. I also partitioned time-series storage and cached expensive reads to keep performance stable as data grows. If I took it further, I would persist all key/quota state for multi-node scale and deepen automated integration coverage around resilience paths.